Module 4: Optimization Strategies

Balaji Srinivasan, Ganapathy Krishnamurthi

Wadhwani School of AI , IIT Madras

Outline of this Module

1. Optimization in Deep Learning
2. A Primer on Convex Optimization
3. Gradient Descent Algorithms
4. Learning Rate Scheduling
5. Accelerated Optimization Methods

Optimization in Deep Learning

Optimization Strategies

Optimization in Deep Learning: The Engine Room

The Story So Far

We have consistently used optimization algorithms to train our deep learning models. They are the tools that:

- Update model parameters iteratively.

Optimization in Deep Learning: The Engine Room

The Story So Far

We have consistently used optimization algorithms to train our deep learning models. They are the tools that:

- Update model parameters iteratively.
- Aim to minimize the value of a **loss function** using a training dataset.

Optimization in Deep Learning: The Engine Room

The Story So Far

We have consistently used optimization algorithms to train our deep learning models. They are the tools that:

- Update model parameters iteratively.
- Aim to minimize the value of a **loss function** using a training dataset.

Is a "Black Box" understanding enough?

Anyone can use optimizers like "SGD" or "Adam" to minimize an objective function.

Optimization in Deep Learning: The Engine Room

The Story So Far

We have consistently used optimization algorithms to train our deep learning models. They are the tools that:

- Update model parameters iteratively.
- Aim to minimize the value of a **loss function** using a training dataset.

Is a "Black Box" understanding enough?

Anyone can use optimizers like "SGD" or "Adam" to minimize an objective function. However, to achieve high performance and efficiency, a **deeper knowledge** is required.

Why Deeper Knowledge is Crucial

Why Deeper Knowledge is Crucial

1. Training Efficiency

- Training a complex model can take hours, days, or even weeks.

Why Deeper Knowledge is Crucial

1. Training Efficiency

- Training a complex model can take hours, days, or even weeks.
- The performance of the optimization algorithm **directly affects** the model's training time.

Why Deeper Knowledge is Crucial

1. Training Efficiency

- Training a complex model can take hours, days, or even weeks.
- The performance of the optimization algorithm **directly affects** the model's training time.
- A better algorithm or better tuning can save significant time and computational resources.

Why Deeper Knowledge is Crucial

1. Training Efficiency

- Training a complex model can take hours, days, or even weeks.
- The performance of the optimization algorithm **directly affects** the model's training time.
- A better algorithm or better tuning can save significant time and computational resources.

2. Model Performance

- Understanding the principles of different optimizers helps in hyperparameter tuning.

Why Deeper Knowledge is Crucial

1. Training Efficiency

- Training a complex model can take hours, days, or even weeks.
- The performance of the optimization algorithm **directly affects** the model's training time.
- A better algorithm or better tuning can save significant time and computational resources.

2. Model Performance

- Understanding the principles of different optimizers helps in hyperparameter tuning.
- This allows us to tune hyperparameters in a **targeted manner**, not just by random guessing.

Why Deeper Knowledge is Crucial

1. Training Efficiency

- Training a complex model can take hours, days, or even weeks.
- The performance of the optimization algorithm **directly affects** the model's training time.
- A better algorithm or better tuning can save significant time and computational resources.

2. Model Performance

- Understanding the principles of different optimizers helps in hyperparameter tuning.
- This allows us to tune hyperparameters in a **targeted manner**, not just by random guessing.
- The result is often a model with better accuracy and generalization.

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.
- **Context:** In our case, this objective is the **training error** (also called empirical risk).

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.
- **Context:** In our case, this objective is the **training error** (also called empirical risk).
- It is concerned with performance on the data it is given.

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.
- **Context:** In our case, this objective is the **training error** (also called empirical risk).
- It is concerned with performance on the data it is given.

Goal of Deep Learning

- **Focus:** Find a model that performs well on unseen data.

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.
- **Context:** In our case, this objective is the **training error** (also called empirical risk).
- It is concerned with performance on the data it is given.

Goal of Deep Learning

- **Focus:** Find a model that performs well on unseen data.
- **Context:** This means minimizing the **generalization error** (also called true risk/risk).

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.
- **Context:** In our case, this objective is the **training error** (also called empirical risk).
- It is concerned with performance on the data it is given.

Goal of Deep Learning

- **Focus:** Find a model that performs well on unseen data.
- **Context:** This means minimizing the **generalization error** (also called true risk/risk).
- It is concerned with statistical inference and prediction.

The Two Goals

Although we use optimization to perform deep learning, their fundamental goals are different.

Goal of Optimization

- **Focus:** Minimize an objective function.
- **Context:** In our case, this objective is the **training error** (also called empirical risk).
- It is concerned with performance on the data it is given.

Goal of Deep Learning

- **Focus:** Find a model that performs well on unseen data.
- **Context:** This means minimizing the **generalization error** (also called true risk/risk).
- It is concerned with statistical inference and prediction.

Primary Goal

Reducing the training error **is a means to an end, not the end itself**. The ultimate goal is to **reduce generalization error**.

Illustration: True Risk vs. Empirical Risk

Illustration: True Risk vs. Empirical Risk

- **True Risk**: The expected loss on the *entire population* of data. This is smooth. We want to find its true minimum.

Illustration: True Risk vs. Empirical Risk

- **True Risk**: The expected loss on the *entire population* of data. This is smooth. We want to find its true minimum.
- **Empirical Risk**: The average loss on our *finite training dataset*. It is often noisy and less smooth. This is what we actually minimize.

Illustration: True Risk vs. Empirical Risk

- **True Risk**: The expected loss on the *entire population* of data. This is smooth. We want to find its true minimum.
- **Empirical Risk**: The average loss on our *finite training dataset*. It is often noisy and less smooth. This is what we actually minimize.

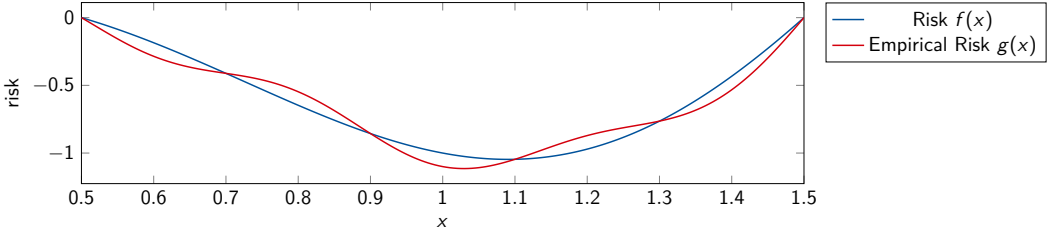
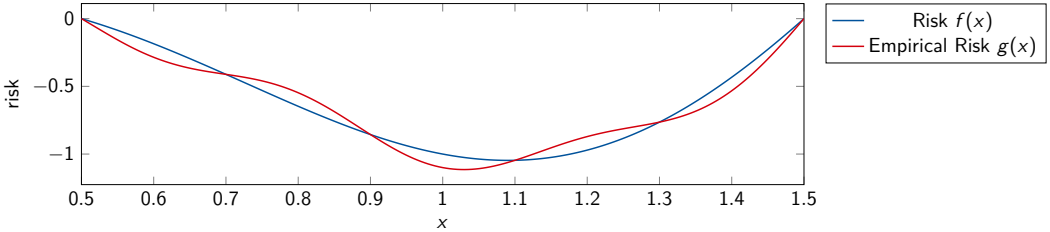


Illustration: True Risk vs. Empirical Risk

- **True Risk**: The expected loss on the *entire population* of data. This is smooth. We want to find its true minimum.
- **Empirical Risk**: The average loss on our *finite training dataset*. It is often noisy and less smooth. This is what we actually minimize.



Key Observation

The minimum of the empirical risk (training error) may be at a different location than the minimum of the true risk (generalization error).

The Balancing Act of Deep Learning

So, training a deep learning model involves two key aspects:

1. **Optimization**: The process of reducing the training error with algorithms like SGD, Adam, etc.

The Balancing Act of Deep Learning

So, training a deep learning model involves two key aspects:

1. **Optimization**: The process of reducing the training error with algorithms like SGD, Adam, etc.
2. **Regularization**: A collection of techniques (e.g., weight decay, dropout) used to control model complexity and reduce the gap between training and generalization error (i.e., fight overfitting).

The Balancing Act of Deep Learning

So, training a deep learning model involves two key aspects:

1. **Optimization**: The process of reducing the training error with algorithms like SGD, Adam, etc.
2. **Regularization**: A collection of techniques (e.g., weight decay, dropout) used to control model complexity and reduce the gap between training and generalization error (i.e., fight overfitting).

Focus of This Module

In this module, we will focus specifically on the **optimization** aspect: the algorithms and challenges involved in minimizing the objective function.

The Landscape of Deep Learning

Key Property: Nonconvexity

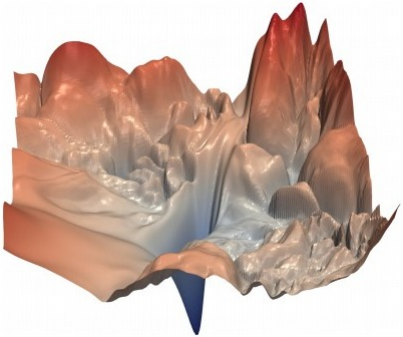
Almost all optimization problems in deep learning are **nonconvex**.

The Landscape of Deep Learning

Key Property: Nonconvexity

Almost all optimization problems in deep learning are **nonconvex**.

This means the "loss landscape" can be incredibly complex, like a rugged mountain range rather than a simple bowl.



Major Optimization Challenges

Optimization Strategies

Major Optimization Challenges

We will explore three of the most significant challenges in deep learning optimization:

Major Optimization Challenges

We will explore three of the most significant challenges in deep learning optimization:

1. **Local Minima:** Getting stuck in a "pretty good" valley instead of the best one.

Major Optimization Challenges

We will explore three of the most significant challenges in deep learning optimization:

1. **Local Minima:** Getting stuck in a "pretty good" valley instead of the best one.
2. **Saddle Points:** Getting stuck on a flat point that is a minimum in one direction but a maximum in another.

Major Optimization Challenges

We will explore three of the most significant challenges in deep learning optimization:

1. **Local Minima:** Getting stuck in a "pretty good" valley instead of the best one.
2. **Saddle Points:** Getting stuck on a flat point that is a minimum in one direction but a maximum in another.
3. **Vanishing Gradients:** Gradients becoming so small that learning effectively stops.

Challenge 1: Local Minima

Definition

- **Local Minimum:** A point x where $f(x)$ is the minimum value over any other points in its immediate vicinity.

Challenge 1: Local Minima

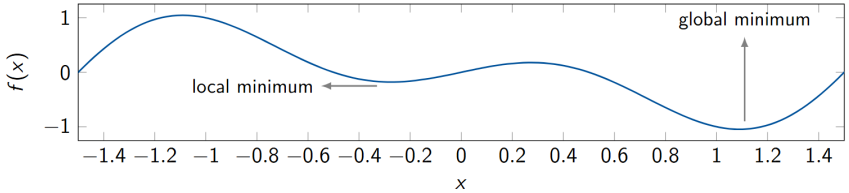
Definition

- **Local Minimum:** A point x where $f(x)$ is the minimum value over any other points in its immediate vicinity.
- **Global Minimum:** A point x where $f(x)$ is the minimum value over the entire domain.

Challenge 1: Local Minima

Definition

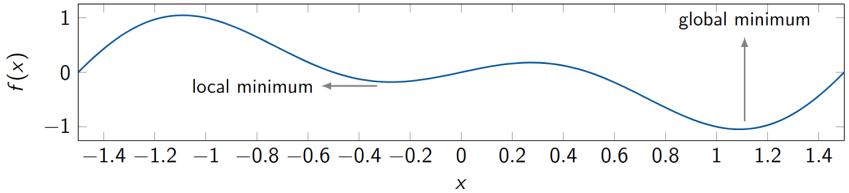
- **Local Minimum:** A point x where $f(x)$ is the minimum value over any other points in its immediate vicinity.
- **Global Minimum:** A point x where $f(x)$ is the minimum value over the entire domain.



Challenge 1: Local Minima

Definition

- **Local Minimum:** A point x where $f(x)$ is the minimum value over any other points in its immediate vicinity.
- **Global Minimum:** A point x where $f(x)$ is the minimum value over the entire domain.



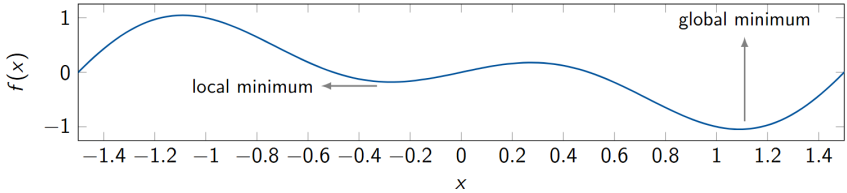
The Problem

An optimization algorithm's updates are based on **local gradient information**.

Challenge 1: Local Minima

Definition

- **Local Minimum:** A point x where $f(x)$ is the minimum value over any other points in its immediate vicinity.
- **Global Minimum:** A point x where $f(x)$ is the minimum value over the entire domain.



The Problem

An optimization algorithm's updates are based on **local gradient information**.It can **converge to a local minimum and stop**, believing it has found the best solution.

Challenge 2: Saddle Points

Definition

A **saddle point** is a location where all gradients of a function vanish (i.e., $\nabla f(x) = 0$), but which is neither a local minimum nor a local maximum.

Challenge 2: Saddle Points

Definition

A **saddle point** is a location where all gradients of a function vanish (i.e., $\nabla f(x) = 0$), but which is neither a local minimum nor a local maximum.

The Problem

Optimization might stall at a saddle point because the gradient is zero, providing no direction for the next step.

Challenge 2: Saddle Points

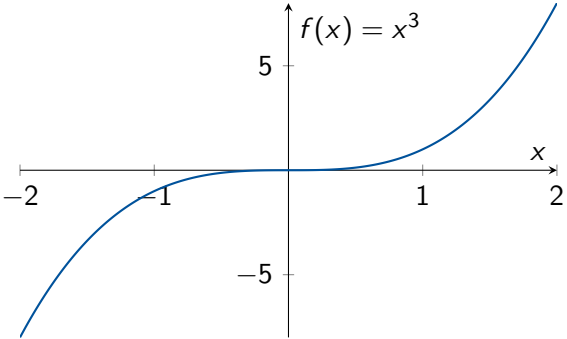
Definition

A **saddle point** is a location where all gradients of a function vanish (i.e., $\nabla f(x) = 0$), but which is neither a local minimum nor a local maximum.

The Problem

Optimization might stall at a saddle point because the gradient is zero, providing no direction for the next step.

Example 1D: $f(x) = x^3$ At $x = 0$, both $f'(0) = 0$ and $f''(0) = 0$. The gradient is zero, but it's clearly not a minimum.



Saddle Points in Higher Dimensions

Saddle Points are More Common than Minima in High Dimensions!

At a zero-gradient point, the type of point depends on the **Hessian matrix** (matrix of second derivatives).

- **Local Minimum**: All eigenvalues of the Hessian are positive.
- **Saddle Point**: Some eigenvalues are positive, and some are negative.

In a high-dimensional space, it's statistically much more likely to get a mix of positive and negative eigenvalues than for all of them to be positive.

Saddle Points in Higher Dimensions

Example 2D:
$f(x, y) = x^2 - y^2$

- At $(0, 0)$, the gradient is $[0, 0]$.
- It's a minimum with respect to x.
- It's a maximum with respect to y.
- The shape is literally a saddle!

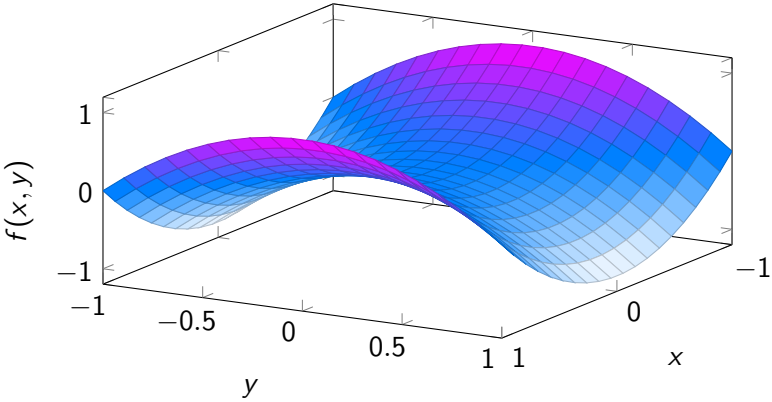


Figure: The function $f(x, y) = x^2 - y^2$ has a saddle point at the origin.

Challenge 3: Vanishing Gradients

Definition

The **vanishing gradient** problem occurs when gradients become infinitesimally small, causing the updates to the model's parameters to become negligible. Learning effectively stops.

Challenge 3: Vanishing Gradients

Definition

The **vanishing gradient** problem occurs when gradients become infinitesimally small, causing the updates to the model's parameters to become negligible. Learning effectively stops.

The Problem

Even if we are not at a minimum or a saddle point, optimization can get stuck for a very long time if the landscape is almost flat.

Challenge 3: Vanishing Gradients

Definition

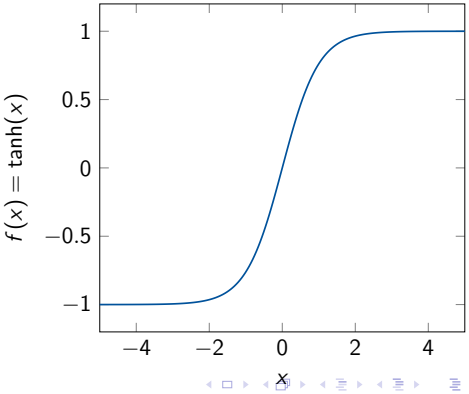
The **vanishing gradient** problem occurs when gradients become infinitesimally small, causing the updates to the model's parameters to become negligible. Learning effectively stops.

The Problem

Even if we are not at a minimum or a saddle point, optimization can get stuck for a very long time if the landscape is almost flat.

Example: $f(x) = \tanh(x)$ The derivative is $f'(x) = 1 - \tanh^2(x)$.

- If we start at $x = 4$, the gradient is $f'(4) \approx 0.0013$. The parameter updates will be tiny, and progress will be extremely slow.



A Primer on Convex Optimization

Optimization Strategies

Why Study Convexity?

A Paradox

The optimization problems in deep learning are generally **nonconvex**. So why dedicate a section to convexity?

Why Study Convexity?

A Paradox

The optimization problems in deep learning are generally **nonconvex**. So why dedicate a section to convexity?

The Reasons

- **Analytical Tractability:** Convex problems are much easier to analyze and understand. They provide a theoretical foundation.

Why Study Convexity?

A Paradox

The optimization problems in deep learning are generally **nonconvex**. So why dedicate a section to convexity?

The Reasons

- **Analytical Tractability:** Convex problems are much easier to analyze and understand. They provide a theoretical foundation.
- **Algorithm Design & Testing:** If an algorithm performs poorly even in a simple convex setting, it is unlikely to work well on complex nonconvex problems. Convexity provides a crucial sanity check.

Why Study Convexity?

A Paradox

The optimization problems in deep learning are generally **nonconvex**. So why dedicate a section to convexity?

The Reasons

- **Analytical Tractability:** Convex problems are much easier to analyze and understand. They provide a theoretical foundation.
- **Algorithm Design & Testing:** If an algorithm performs poorly even in a simple convex setting, it is unlikely to work well on complex nonconvex problems. Convexity provides a crucial sanity check.
- **Local Properties:** Even though the global loss landscape is nonconvex, it often exhibits convex-like properties near a local minimum. This insight can lead to better optimization algorithms.

Convex Sets: The Foundation

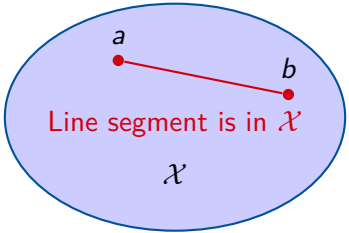
Definition

A set $\mathcal{X}$ in a vector space is **convex** if for any two points $a, b \in \mathcal{X}$, the entire line segment connecting a and b is also in $\mathcal{X}$.

Convex Sets: The Foundation

Definition

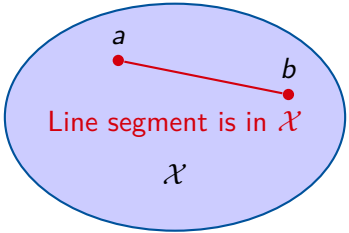
A set $\mathcal{X}$ in a vector space is **convex** if for any two points $a, b \in \mathcal{X}$, the entire line segment connecting a and b is also in $\mathcal{X}$.



Convex Sets: The Foundation

Definition

A set $\mathcal{X}$ in a vector space is **convex** if for any two points $a, b \in \mathcal{X}$, the entire line segment connecting a and b is also in $\mathcal{X}$.

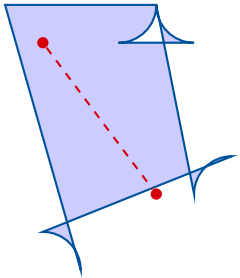


Mathematical Formulation

For all $\lambda \in [0, 1]$, we have:

$$\lambda a + (1 - \lambda) b \in \mathcal{X} \quad \text{whenever } a, b \in \mathcal{X}$$

Examples: Convex vs. Nonconvex Sets



Examples: Convex vs. Nonconvex Sets

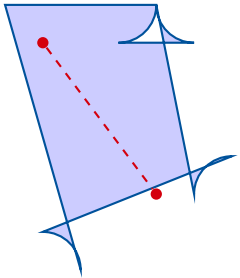
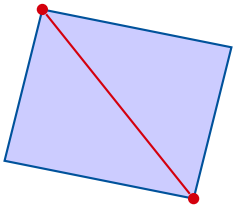


Figure: **Nonconvex.** The line segment goes outside the set.



Examples: Convex vs. Nonconvex Sets

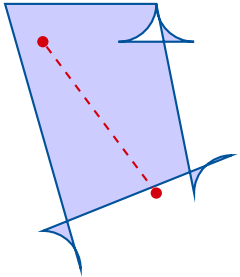


Figure: **Nonconvex.** The line segment goes outside the set.

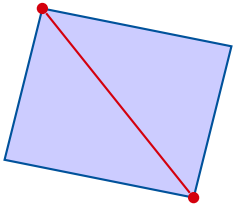
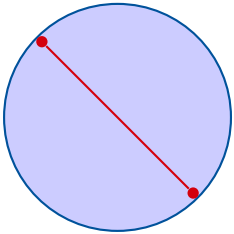


Figure: **Convex.** All line segments are contained.



Examples: Convex vs. Nonconvex Sets

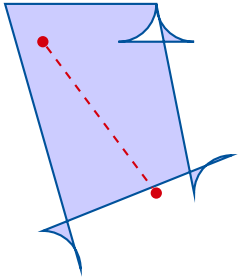


Figure: **Nonconvex.** The line segment goes outside the set.

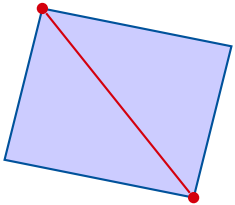


Figure: **Convex.** All line segments are contained.

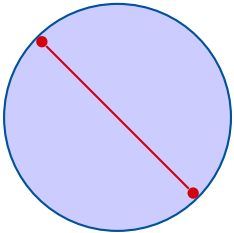


Figure: **Convex.** The disk is a classic convex set.

Properties of Convex Sets

Intersection

The intersection of two (or any number of) convex sets is also **convex**.

Properties of Convex Sets

Intersection

The intersection of two (or any number of) convex sets is also **convex**.

Proof Sketch: If points a, b are in the intersection $\mathcal{X} \cap \mathcal{Y}$, they are in $\mathcal{X}$ and they are in $\mathcal{Y}$. Since both sets are convex, the line segment connecting a, b must be in $\mathcal{X}$ and in $\mathcal{Y}$. Therefore, the segment must be in their intersection.

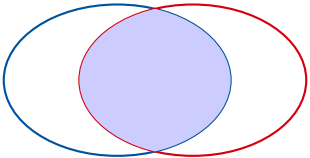


Figure: Intersection of two convex sets is convex.

Properties of Convex Sets

Union

The union of two convex sets is **not necessarily** convex.

Properties of Convex Sets

Union

The union of two convex sets is **not necessarily** convex.

Counter example: Consider two disjoint convex sets $\mathcal{X}$ and $\mathcal{Y}$. Pick a point $a \in \mathcal{X}$ and $b \in \mathcal{Y}$. The line segment between them must contain points that are in neither set.

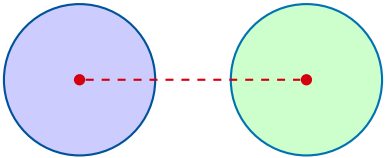


Figure: Union of two convex sets may not be convex.

Convex Functions

Definition

Given a convex set $\mathcal{X}$, a function $f : \mathcal{X} \to \mathbb{R}$ is **convex** if for all $x, x' \in \mathcal{X}$ and for all $\lambda \in [0, 1]$ we have:

$$\lambda f(x) + (1 - \lambda) f(x') \geq f(\lambda x + (1 - \lambda) x')$$

Convex Functions

Definition

Given a convex set $\mathcal{X}$, a function $f : \mathcal{X} \to \mathbb{R}$ is **convex** if for all $x, x' \in \mathcal{X}$ and for all $\lambda \in [0, 1]$ we have:

$$\lambda f(x) + (1 - \lambda) f(x') \geq f(\lambda x + (1 - \lambda) x')$$

Visual Intuition

The line segment connecting any two points on the function's graph (the *chord*) lies **at or above** the graph of the function.

Convex Functions

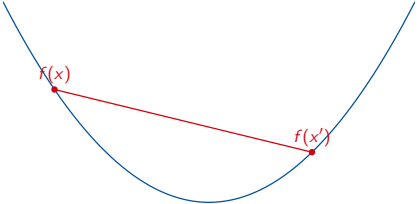
Definition

Given a convex set $\mathcal{X}$, a function $f : \mathcal{X} \to \mathbb{R}$ is **convex** if for all $x, x' \in \mathcal{X}$ and for all $\lambda \in [0, 1]$ we have:

$$\lambda f(x) + (1 - \lambda) f(x') \geq f(\lambda x + (1 - \lambda) x')$$

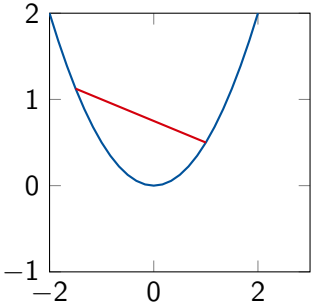
Visual Intuition

The line segment connecting any two points on the function's graph (the *chord*) lies **at or above** the graph of the function.



Examples of Functions

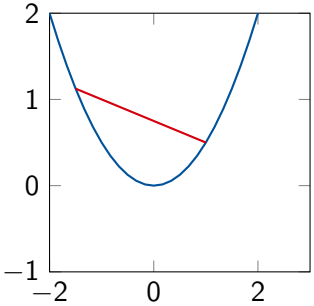
$f(x) = 0.5x^2$



Convex

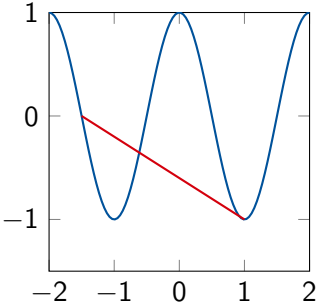
Examples of Functions

$f(x) = 0.5x^2$



Convex

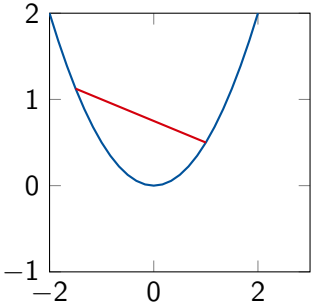
$g(x) = \cos(\pi x)$



Nonconvex

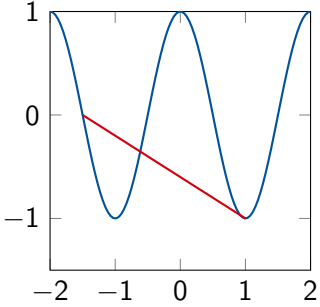
Examples of Functions

$f(x) = 0.5x^2$



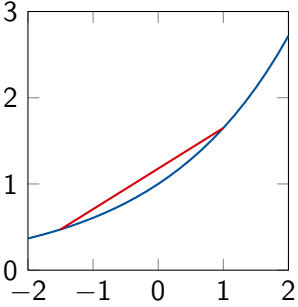
Convex

$g(x) = \cos(\pi x)$



Nonconvex

$h(x) = \exp(0.5x)$



Convex

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Proof by Contradiction

1. Assume x^* is a local minimum, but *not* a global minimum.

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Proof by Contradiction

1. Assume x^* is a local minimum, but *not* a global minimum.
2. This means there must exist some other point x' where $f(x') < f(x^*)$.

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Proof by Contradiction

1. Assume x^* is a local minimum, but *not* a global minimum.
2. This means there must exist some other point x' where $f(x') < f(x^*)$.
3. Consider a point on the line segment between x^* and x', very close to x^*: $z = \lambda x' + (1 - \lambda)x^*$ for a small $\lambda > 0$.

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Proof by Contradiction

1. Assume x^* is a local minimum, but *not* a global minimum.
2. This means there must exist some other point x' where $f(x') < f(x^*)$.
3. Consider a point on the line segment between x^* and x', very close to x^*: $z = \lambda x' + (1 - \lambda)x^*$ for a small $\lambda > 0$.
4. By the definition of convexity: $f(z) \leq \lambda f(x') + (1 - \lambda)f(x^*) < \lambda f(x^*) + (1 - \lambda)f(x^*) = f(x^*)$.

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Proof by Contradiction

1. Assume x^* is a local minimum, but *not* a global minimum.
2. This means there must exist some other point x' where $f(x') < f(x^*)$.
3. Consider a point on the line segment between x^* and x', very close to x^*: $z = \lambda x' + (1 - \lambda)x^*$ for a small $\lambda > 0$.
4. By the definition of convexity: $f(z) \leq \lambda f(x') + (1 - \lambda)f(x^*) < \lambda f(x^*) + (1 - \lambda)f(x^*) = f(x^*)$.
5. So we found a point z arbitrarily close to x^* with $f(z) < f(x^*)$. This contradicts the assumption that x^* is a local minimum!

Key Property 1: Local Minima Are Global Minima

Theorem

For a convex function f, any local minimum is also a global minimum.

Proof by Contradiction

1. Assume x^* is a local minimum, but *not* a global minimum.
2. This means there must exist some other point x' where $f(x') < f(x^*)$.
3. Consider a point on the line segment between x^* and x', very close to x^*: $z = \lambda x' + (1 - \lambda)x^*$ for a small $\lambda > 0$.
4. By the definition of convexity: $f(z) \leq \lambda f(x') + (1 - \lambda)f(x^*) < \lambda f(x^*) + (1 - \lambda)f(x^*) = f(x^*)$.
5. So we found a point z arbitrarily close to x^* with $f(z) < f(x^*)$. This contradicts the assumption that x^* is a local minimum!
6. Therefore, the initial assumption must be false. x^* must be a global minimum.

Key Property 2: Second Derivatives

The Second Derivative Test

- **1D Function $f : \mathbb{R} \to \mathbb{R}$:** f is convex if and only if its second derivative is non-negative, $f''(x) \geq 0$, for all x.

Key Property 2: Second Derivatives

The Second Derivative Test

- **1D Function $f : \mathbb{R} \to \mathbb{R}$:** f is convex if and only if its second derivative is non-negative, $f''(x) \geq 0$, for all x.
  Intuition: The function is always curving upwards (or is flat).

Key Property 2: Second Derivatives

The Second Derivative Test

- **1D Function** $f : \mathbb{R} \to \mathbb{R}$**:** f is convex if and only if its second derivative is non-negative, $f''(x) \geq 0$, for all x.
  Intuition: The function is always curving upwards (or is flat).
- **Multidimensional Function** $f : \mathbb{R}^n \to \mathbb{R}$**:** f is convex if and only if its **Hessian matrix** $\mathbf{H} = \nabla^2 f(x)$ is positive semidefinite for all x.

Key Property 2: Second Derivatives

The Second Derivative Test

- **1D Function** $f : \mathbb{R} \to \mathbb{R}$**:** f is convex if and only if its second derivative is non-negative, $f''(x) \geq 0$, for all x.
  Intuition: The function is always curving upwards (or is flat).
- **Multidimensional Function** $f : \mathbb{R}^n \to \mathbb{R}$**:** f is convex if and only if its **Hessian matrix** $\mathbf{H} = \nabla^2 f(x)$ is positive semidefinite for all x.
  A matrix $\mathbf{H}$ *is positive semidefinite if* $\mathbf{z}^T \mathbf{H} \mathbf{z} \geq 0$ *for all vectors* $\mathbf{z}$*. This is equivalent to all its eigenvalues being non-negative.*

Key Property 2: Second Derivatives

The Second Derivative Test

- **1D Function $f : \mathbb{R} \to \mathbb{R}$:** f is convex if and only if its second derivative is non-negative, $f''(x) \geq 0$, for all x.
  Intuition: The function is always curving upwards (or is flat).
- **Multidimensional Function $f : \mathbb{R}^n \to \mathbb{R}$:** f is convex if and only if its **Hessian matrix** $\mathbf{H} = \nabla^2 f(x)$ is positive semidefinite for all x.
  A matrix $\mathbf{H}$ is positive semidefinite if $\mathbf{z}^T \mathbf{H} \mathbf{z} \geq 0$ for all vectors $\mathbf{z}$. This is equivalent to all its eigenvalues being non-negative.

Example: $f(\mathbf{x}) = \frac{1}{2}||\mathbf{x}||^2 = \frac{1}{2}\sum_i x_i^2$

- Gradient: $\nabla f(\mathbf{x}) = \mathbf{x}$
- Hessian: $\nabla^2 f(\mathbf{x}) = \mathbf{I}$ (the identity matrix)
- The eigenvalues of $\mathbf{I}$ are equal to 1. Since they are all positive, the function is convex.

Jensen's Inequality: A Powerful Tool

Jensen's Inequality

For a convex function f, non-negative weights α_i such that $\sum \alpha_i = 1$, and a random variable X:

1. **Discrete form**

$$\sum_i \alpha_i f(x_i) \geq f\left(\sum_i \alpha_i x_i\right)$$

Jensen's Inequality: A Powerful Tool

Jensen's Inequality

For a convex function f, non-negative weights α_i such that $\sum \alpha_i = 1$, and a random variable X:

1. **Discrete form**

$$\sum_i \alpha_i f(x_i) \geq f\left(\sum_i \alpha_i x_i\right)$$

2. **Continuous form**

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$$

Jensen's Inequality: A Powerful Tool

Jensen's Inequality

For a convex function f, non-negative weights α_i such that $\sum \alpha_i = 1$, and a random variable X:

1. **Discrete form**

$$\sum_i \alpha_i f(x_i) \geq f\left(\sum_i \alpha_i x_i\right)$$

2. **Continuous form**

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$$

Intuition

The average of the function's values is greater than or equal to the function evaluated at the average input.

Handling Constraints

Convex optimization provides powerful tools for handling constraints efficiently.

Handling Constraints

Convex optimization provides powerful tools for handling constraints efficiently.

Constrained Optimization Problem

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & c_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \ldots, n \end{aligned}$$

where f (the objective) and c_i (the constraints) are convex functions.

Handling Constraints

Convex optimization provides powerful tools for handling constraints efficiently.

Constrained Optimization Problem

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & c_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \ldots, n \end{aligned}$$

where f (the objective) and c_i (the constraints) are convex functions.

How can we solve this?

There are several strategies. We will look at three common ones:

Handling Constraints

Convex optimization provides powerful tools for handling constraints efficiently.

Constrained Optimization Problem

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & c_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \ldots, n \end{aligned}$$

where f (the objective) and c_i (the constraints) are convex functions.

How can we solve this?

There are several strategies. We will look at three common ones:

1. The Lagrangian

Handling Constraints

Convex optimization provides powerful tools for handling constraints efficiently.

Constrained Optimization Problem

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & c_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \ldots, n \end{aligned}$$

where f (the objective) and c_i (the constraints) are convex functions.

How can we solve this?

There are several strategies. We will look at three common ones:

1. The Lagrangian
2. Penalties

Handling Constraints

Convex optimization provides powerful tools for handling constraints efficiently.

Constrained Optimization Problem

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & c_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, \ldots, n \end{aligned}$$

where f (the objective) and c_i (the constraints) are convex functions.

How can we solve this?

There are several strategies. We will look at three common ones:

1. The Lagrangian
2. Penalties
3. Projections

Constraints: The Lagrangian

The Lagrangian method converts a constrained problem into an unconstrained one.

The Lagrangian

We define the Lagrangian function L by introducing new variables $\alpha_i \geq 0$ called **Lagrange multipliers**:

$$L(\mathbf{x}, \alpha_1, \ldots, \alpha_n) = f(\mathbf{x}) + \sum_{i=1}^{n} \alpha_i c_i(\mathbf{x})$$

Constraints: The Lagrangian

The Saddle-Point Problem

Solving the original problem is equivalent to finding a saddle point of L:

$$\max_{\alpha_i \geq 0} \min_{\mathbf{x}} L(\mathbf{x}, \alpha)$$

- We **minimize** with respect to the original variables $\mathbf{x}$.
- We **maximize** with respect to the multipliers α.

Constraints: The Lagrangian

The Saddle-Point Problem

Solving the original problem is equivalent to finding a saddle point of L:

$$\max_{\alpha_i \geq 0} \min_{\mathbf{x}} L(\mathbf{x}, \alpha)$$

- We **minimize** with respect to the original variables $\mathbf{x}$.
- We **maximize** with respect to the multipliers α.

Intuition: The multipliers α_i act as "prices" or "forces". If a constraint $c_i(\mathbf{x}) \leq 0$ is violated (i.e., $c_i(\mathbf{x}) > 0$), we can increase α_i to increase the value of L, pushing $\mathbf{x}$ back towards the feasible region. If a constraint is satisfied ($c_i(\mathbf{x}) < 0$), the optimal choice is $\alpha_i = 0$.

Constraints: Penalties

Penalties

Instead of formally solving the saddle-point problem, we can add a penalty term to the objective.

$$\text{minimize } f(\mathbf{x}) + \sum_{i=1}^{n} \alpha_i c_i(\mathbf{x})$$

This is an *approximate* way to satisfy constraints.

Weight Decay

Weight decay is a penalty method. Adding $\frac{\lambda}{2}||\mathbf{w}||^2$ to the loss is equivalent to enforcing a constraint that $||\mathbf{w}||^2 - r^2 \leq 0$ for some radius r.

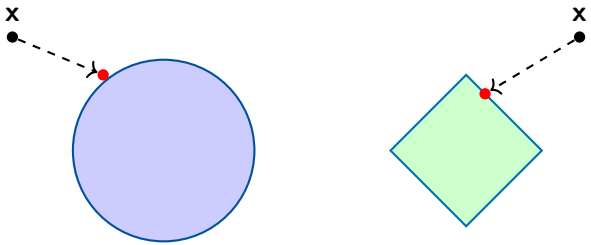
Constraints: Projections

Projections

An alternative is to take an optimization step and then "project" the result back into the feasible set.

$$\text{Proj}_{\mathcal{X}}(\mathbf{x}) = \underset{\mathbf{x}' \in \mathcal{X}}{\text{argmin}} ||\mathbf{x} - \mathbf{x}'||$$

This finds the closest point in the convex set $\mathcal{X}$ to the point $\mathbf{x}$.



Put it together

Local Minima are Global Minima

For a convex function, any local minimum is also a global minimum. This means we cannot "get stuck" in a suboptimal valley.

Put it together

Local Minima are Global Minima

For a convex function, any local minimum is also a global minimum. This means we cannot "get stuck" in a suboptimal valley.

Second Derivative Condition

A twice-differentiable function f is convex if and only if its second derivative (Hessian matrix) is positive semidefinite. This condition relates convexity to the function's curvature.

Put it together

Local Minima are Global Minima

For a convex function, any local minimum is also a global minimum. This means we cannot "get stuck" in a suboptimal valley.

Second Derivative Condition

A twice-differentiable function f is convex if and only if its second derivative (Hessian matrix) is positive semidefinite. This condition relates convexity to the function's curvature.

Jensen's Inequality

A cornerstone of convex analysis. For a convex function f and a random variable X:

$$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$$

The expectation of the function is greater than or equal to the function of the expectation.

Gradient Descent Algorithms

Optimization Strategies

Gradient Descent: The Core Idea

Taylor Expansion

For a small displacement ϵ, we can approximate the function value using a first-order Taylor expansion:

$$f(\mathbf{x} + \epsilon) \approx f(\mathbf{x}) + \epsilon^T \nabla f(\mathbf{x})$$

Gradient Descent: The Core Idea

Taylor Expansion

For a small displacement ϵ, we can approximate the function value using a first-order Taylor expansion:

$$f(\mathbf{x} + \epsilon) \approx f(\mathbf{x}) + \epsilon^T \nabla f(\mathbf{x})$$

Which direction to move?

- To make $f(\mathbf{x} + \epsilon)$ as small as possible, we need to make the term $\epsilon^T \nabla f(\mathbf{x})$ as negative as possible.

Gradient Descent: The Core Idea

Taylor Expansion

For a small displacement ϵ, we can approximate the function value using a first-order Taylor expansion:

$$f(\mathbf{x} + \epsilon) \approx f(\mathbf{x}) + \epsilon^T \nabla f(\mathbf{x})$$

Which direction to move?

- To make $f(\mathbf{x} + \epsilon)$ as small as possible, we need to make the term $\epsilon^T \nabla f(\mathbf{x})$ as negative as possible.
- The direction that maximally decreases the value is the direction opposite to the gradient, i.e., $-\nabla f(\mathbf{x})$.

Gradient Descent: The Core Idea

Taylor Expansion

For a small displacement ϵ, we can approximate the function value using a first-order Taylor expansion:

$$f(\mathbf{x} + \epsilon) \approx f(\mathbf{x}) + \epsilon^T \nabla f(\mathbf{x})$$

Which direction to move?

- To make $f(\mathbf{x} + \epsilon)$ as small as possible, we need to make the term $\epsilon^T \nabla f(\mathbf{x})$ as negative as possible.
- The direction that maximally decreases the value is the direction opposite to the gradient, i.e., $-\nabla f(\mathbf{x})$.
- So, we should choose our displacement ϵ to be proportional to the negative gradient:

$$\epsilon = -\eta \nabla f(\mathbf{x})$$

Gradient Descent: The Core Idea

Taylor Expansion

For a small displacement ϵ, we can approximate the function value using a first-order Taylor expansion:

$$f(\mathbf{x} + \epsilon) \approx f(\mathbf{x}) + \epsilon^T \nabla f(\mathbf{x})$$

Which direction to move?

- To make $f(\mathbf{x} + \epsilon)$ as small as possible, we need to make the term $\epsilon^T \nabla f(\mathbf{x})$ as negative as possible.
- The direction that maximally decreases the value is the direction opposite to the gradient, i.e., $-\nabla f(\mathbf{x})$.
- So, we should choose our displacement ϵ to be proportional to the negative gradient:

$$\epsilon = -\eta \nabla f(\mathbf{x})$$

Here, $\eta > 0$ is a small positive scalar called the **learning rate**.

The Gradient Descent Algorithm

This leads to the fundamental update rule for Gradient Descent (GD).

Theorem (Gradient Descent Algorithm)

Starting with an initial value $\mathbf{x}_0$, we iteratively update the parameters using the rule:

$$\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$$

We repeat this until a stopping condition is met (e.g., gradient is small, or max iterations reached).

The Gradient Descent Algorithm

1D Example: $f(x) = x^2$

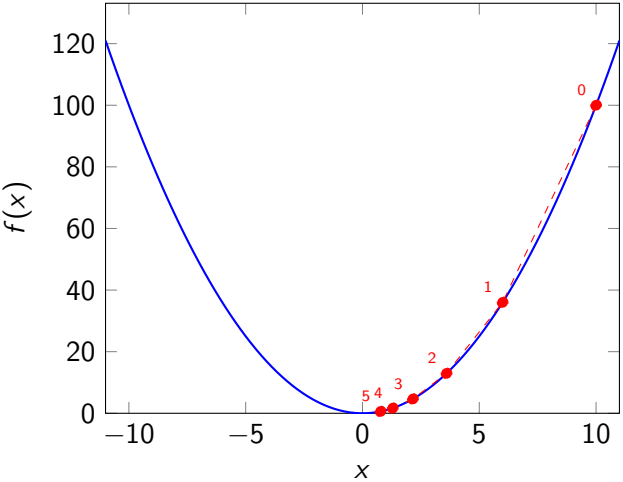
- Objective: $f(x) = x^2$
- Derivative: $f'(x) = 2x$
- Update Rule: $x_{t+1} \leftarrow x_t - \eta(2x_t)$

The Gradient Descent Algorithm

1D Example: $f(x) = x^2$

- Objective: $f(x) = x^2$
- Derivative: $f'(x) = 2x$
- Update Rule: $x_{t+1} \leftarrow x_t - \eta(2x_t)$

Let's visualize this. We start at $x = 10$ with a learning rate $\eta = 0.2$.

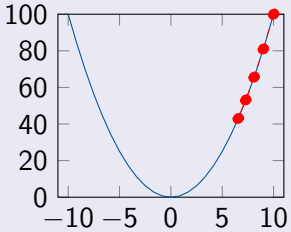


The Importance of the Learning Rate (η)

The choice of learning rate is critical for gradient descent to succeed.

Case 1: η is too small (e.g., 0.05)

- Progress is very slow.
- Requires many iterations to converge.

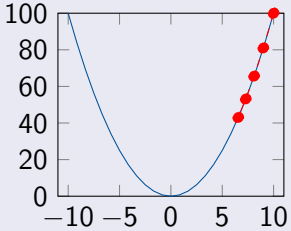


The Importance of the Learning Rate (η)

The choice of learning rate is critical for gradient descent to succeed.

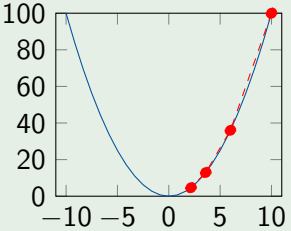
Case 1: η is too small (e.g., 0.05)

- Progress is very slow.
- Requires many iterations to converge.



Case 2: η is just right (e.g., 0.2)

- Makes reasonable progress each step.
- Converges efficiently.

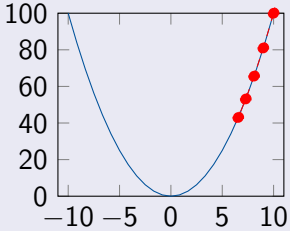


The Importance of the Learning Rate (η)

The choice of learning rate is critical for gradient descent to succeed.

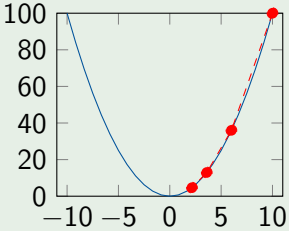
Case 1: η is too small (e.g., 0.05)

- Progress is very slow.
- Requires many iterations to converge.



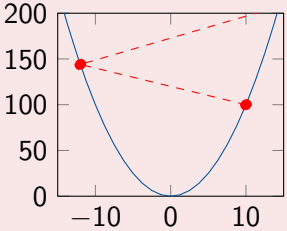
Case 2: η is just right (e.g., 0.2)

- Makes reasonable progress each step.
- Converges efficiently.



Case 3: η is too large (e.g., 1.1)

- "Overshoots" the minimum.
- Leads to divergence.



Multivariate Gradient Descent

Let's move to a more realistic scenario with a 2D objective function: $f(\mathbf{x}) = x_1^2 + 2x_2^2$

Multivariate Gradient Descent

Let's move to a more realistic scenario with a 2D objective function: $f(\mathbf{x}) = x_1^2 + 2x_2^2$

Gradient

The gradient is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

Multivariate Gradient Descent

Let's move to a more realistic scenario with a 2D objective function: $f(\mathbf{x}) = x_1^2 + 2x_2^2$

Gradient

The gradient is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

The Problem: Bad-Conditioning

Notice that the function is steeper in the x_2 direction than in the x_1 direction. The "level sets" (contours) are ellipses, not circles. This makes finding a good single learning rate difficult.

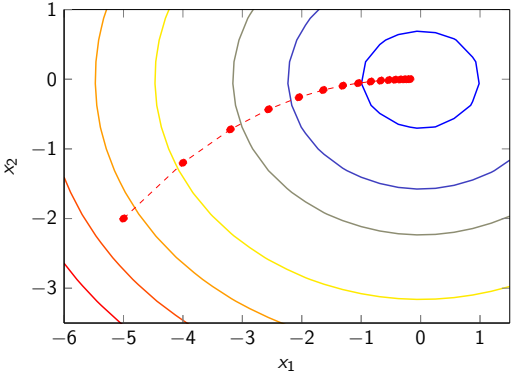
Multivariate Gradient Descent

Let's move to a more realistic scenario with a 2D objective function: $f(\mathbf{x}) = x_1^2 + 2x_2^2$

Gradient

The gradient is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$



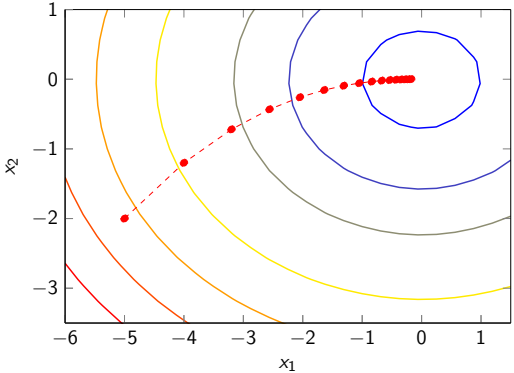
Multivariate Gradient Descent

Let's move to a more realistic scenario with a 2D objective function: $f(\mathbf{x}) = x_1^2 + 2x_2^2$

Gradient

The gradient is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$



The Problem: Bad-Conditioning

Notice that the function is steeper in the x_2 direction than in the x_1 direction. The "level sets" (contours) are ellipses, not circles. This makes finding a good single learning rate difficult.

Adaptive Methods: Newton's Method

Gradient descent only uses first-order (gradient) information. Can we do better by using second-order (curvature) information?

Adaptive Methods: Newton's Method

Gradient descent only uses first-order (gradient) information. Can we do better by using second-order (curvature) information?

Second-Order Taylor Expansion

$$f(\mathbf{x} + \boldsymbol{\epsilon}) \approx f(\mathbf{x}) + \boldsymbol{\epsilon}^T \nabla f(\mathbf{x}) + \frac{1}{2}\boldsymbol{\epsilon}^T \mathbf{H} \boldsymbol{\epsilon}$$

where $\mathbf{H} = \nabla^2 f(\mathbf{x})$ is the **Hessian matrix** of second derivatives.

Adaptive Methods: Newton's Method

Gradient descent only uses first-order (gradient) information. Can we do better by using second-order (curvature) information?

Second-Order Taylor Expansion

$$f(\mathbf{x}+\boldsymbol{\epsilon}) \approx f(\mathbf{x}) + \boldsymbol{\epsilon}^T \nabla f(\mathbf{x}) + \frac{1}{2}\boldsymbol{\epsilon}^T \mathbf{H}\boldsymbol{\epsilon}$$

where $\mathbf{H} = \nabla^2 f(\mathbf{x})$ is the **Hessian matrix** of second derivatives.

Finding the Minimum

To find the minimum of this quadratic approximation, we take the derivative with respect to $\boldsymbol{\epsilon}$ and set it to zero:

$$\nabla f(\mathbf{x}) + \mathbf{H}\boldsymbol{\epsilon} = 0 \implies \boldsymbol{\epsilon} = -\mathbf{H}^{-1}\nabla f(\mathbf{x})$$

Adaptive Methods: Newton's Method

Gradient descent only uses first-order (gradient) information. Can we do better by using second-order (curvature) information?

Second-Order Taylor Expansion

$$f(\mathbf{x}+\boldsymbol{\epsilon}) \approx f(\mathbf{x}) + \boldsymbol{\epsilon}^T \nabla f(\mathbf{x}) + \frac{1}{2}\boldsymbol{\epsilon}^T \mathbf{H} \boldsymbol{\epsilon}$$

where $\mathbf{H} = \nabla^2 f(\mathbf{x})$ is the **Hessian matrix** of second derivatives.

Finding the Minimum

To find the minimum of this quadratic approximation, we take the derivative with respect to $\boldsymbol{\epsilon}$ and set it to zero:

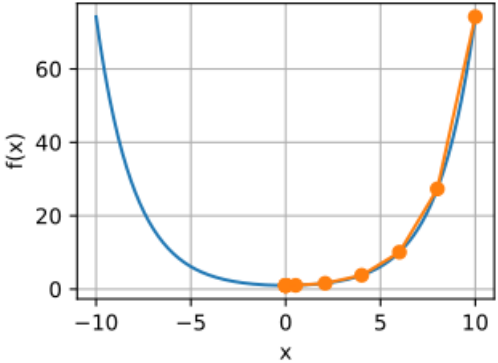
$$\nabla f(\mathbf{x}) + \mathbf{H}\boldsymbol{\epsilon} = 0 \implies \boldsymbol{\epsilon} = -\mathbf{H}^{-1}\nabla f(\mathbf{x})$$

This gives the update rule for **Newton's method**. It uses the inverse Hessian to rescale the gradient, effectively creating a specialized learning rate for each direction.

Newton's Method: The Good and The Bad

The Good: Fast Convergence

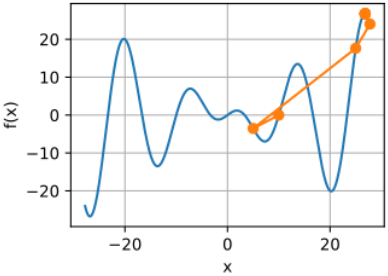
For "nice" (e.g., strongly convex) functions, Newton's method can converge extremely quickly, often in a handful of steps. For a quadratic function, it converges in a **single step**!



Newton's Method: The Good and The Bad

The Bad: Non-convexity & Cost

- If the function is not convex, the Hessian $\mathbf{H}$ can have negative eigenvalues. In this case, Newton's method can actually move towards a **maximum**, causing divergence.
- **High Cost:** Computing, storing, and inverting the Hessian is computationally prohibitive for deep learning models. If a model has d parameters, the Hessian is $d \times d$, which requires $O(d^2)$ space and $O(d^3)$ for inversion.



Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

The Gradient Calculation

The "true" or "batch" gradient is therefore also an average: $\nabla f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x})$

Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

The Gradient Calculation

The "true" or "batch" gradient is therefore also an average: $\nabla f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x})$

The Computational Bottleneck

Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

The Gradient Calculation

The "true" or "batch" gradient is therefore also an average: $\nabla f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x})$

The Computational Bottleneck

- To compute this gradient, we must perform a **full pass** over the entire training dataset.

Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

The Gradient Calculation

The "true" or "batch" gradient is therefore also an average: $\nabla f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x})$

The Computational Bottleneck

- To compute this gradient, we must perform a **full pass** over the entire training dataset.
- The computational cost for a single gradient update is $O(n)$.

Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

The Gradient Calculation

The "true" or "batch" gradient is therefore also an average: $\nabla f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x})$

The Computational Bottleneck

- To compute this gradient, we must perform a **full pass** over the entire training dataset.
- The computational cost for a single gradient update is $O(n)$.
- If n is in the millions or billions (common today), this is extremely slow and inefficient.

Batch Gradient Descent

In deep learning, the objective function is usually an average over a training dataset of n examples: $f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} f_i(\mathbf{x})$ where $f_i(\mathbf{x})$ is the loss for the i-th example.

The Gradient Calculation

The "true" or "batch" gradient is therefore also an average: $\nabla f(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x})$

The Computational Bottleneck

- To compute this gradient, we must perform a **full pass** over the entire training dataset.
- The computational cost for a single gradient update is $O(n)$.
- If n is in the millions or billions (common today), this is extremely slow and inefficient.
- Much of the data in a large dataset is redundant; we shouldn't need to see all of it to make a small step.

Stochastic Gradient Descent (SGD): The Solution

The Core Idea

Instead of computing the gradient on the whole dataset, what if we just pick one example at random and compute the gradient on that?

Stochastic Gradient Descent (SGD): The Solution

The Core Idea

Instead of computing the gradient on the whole dataset, what if we just pick one example at random and compute the gradient on that?

Theorem (Stochastic Gradient Descent Algorithm)

At each iteration t:

1. *Uniformly sample an index $i_t \in \{1, \ldots, n\}$.*
2. *Compute the* ***stochastic gradient*** $\nabla f_{i_t}(\mathbf{x}_t)$.
3. *Update the parameters:*

$$\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t - \eta \nabla f_{i_t}(\mathbf{x}_t)$$

Stochastic Gradient Descent (SGD): The Solution

Key Advantages

- The computational cost per update drops from $O(n)$ to $O(1)$. This is a massive speedup!

Stochastic Gradient Descent (SGD): The Solution

Key Advantages

- The computational cost per update drops from $O(n)$ to $O(1)$. This is a massive speedup!
- **Why does this work?** The stochastic gradient is an *unbiased estimate* of the true gradient:

$$\mathbb{E}_{i_t}[\nabla f_{i_t}(\mathbf{x})] = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(\mathbf{x}) = \nabla f(\mathbf{x})$$

  On average, we are moving in the right direction.

Visualizing SGD vs. GD

Observations

- **GD Path (Blue):** Smooth, deterministic, directly moves towards the minimum.
- **SGD Path (Red):** Noisy, stochastic. Each step is a rough approximation.
- Even when near the minimum, SGD continues to bounce around due to the gradient noise.

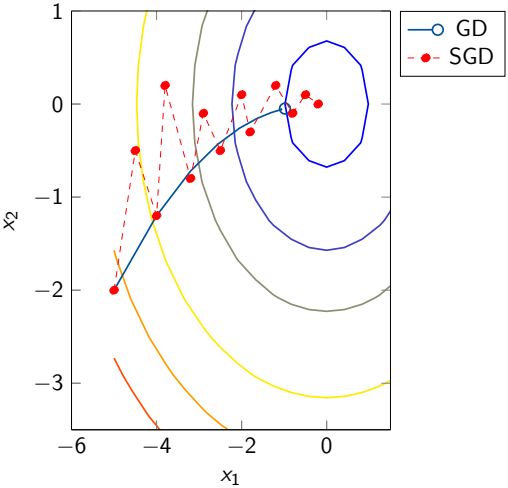
Visualizing SGD vs. GD

Observations

- **GD Path (Blue):** Smooth, deterministic, directly moves towards the minimum.
- **SGD Path (Red):** Noisy, stochastic. Each step is a rough approximation.
- Even when near the minimum, SGD continues to bounce around due to the gradient noise.

The Learning Rate in SGD

This inherent noise means we cannot use a large, constant learning rate. To converge, the learning rate η must be decreased over time.



Finding the Middle Ground

We have seen two extremes:

Finding the Middle Ground

We have seen two extremes:

Batch Gradient Descent

- ✓ Uses entire dataset; accurate gradient.
- ✓ Smooth, stable convergence.
- ✗ Computationally expensive ($O(n)$).
- ✗ Inefficient for redundant data.

Finding the Middle Ground

We have seen two extremes:

Batch Gradient Descent

- ✓ Uses entire dataset; accurate gradient.
- ✓ Smooth, stable convergence.
- ✗ Computationally expensive ($O(n)$).
- ✗ Inefficient for redundant data.

Stochastic Gradient Descent

- ✓ Uses one example; very cheap ($O(1)$).
- ✗ Noisy gradient estimates.
- ✗ Requires careful LR scheduling.
- ✗ **Does not leverage modern parallel hardware!**

Finding the Middle Ground

We have seen two extremes:

Batch Gradient Descent

- ✓ Uses entire dataset; accurate gradient.
- ✓ Smooth, stable convergence.
- ✗ Computationally expensive ($O(n)$).
- ✗ Inefficient for redundant data.

Stochastic Gradient Descent

- ✓ Uses one example; very cheap ($O(1)$).
- ✗ Noisy gradient estimates.
- ✗ Requires careful LR scheduling.
- ✗ **Does not leverage modern parallel hardware!**

The Solution: Minibatch Stochastic Gradient Descent

Instead of one example or all examples, we use a small batch of examples. This is the de facto standard for training deep neural networks.

The Power of Vectorization

The primary motivation for minibatches is computational efficiency. Modern CPUs and GPUs are parallel processors designed to perform the same operation parallely

The Power of Vectorization

The primary motivation for minibatches is computational efficiency. Modern CPUs and GPUs are parallel processors designed to perform the same operation parallely

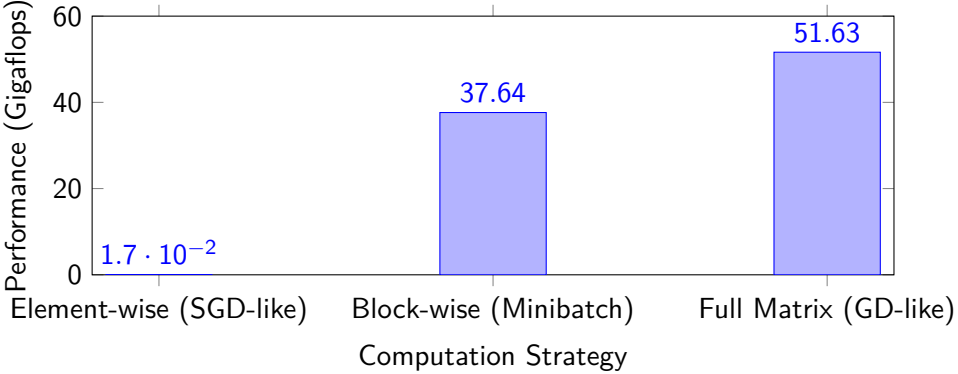
Matrix Multiplication Example

Consider computing $\mathbf{A} = \mathbf{BC}$ where matrices are 256×256.

- **SGD-like (Element by element):** Computing each element A_{ij} via a dot product is extremely slow. It requires many separate, small memory accesses.
- **Minibatch-like (Block by block):** Multiplying blocks of the matrices is much faster as it keeps data in fast cache memory.
- **GD-like (Full matrix):** Performing the entire operation at once is the fastest, as it can be handed off to a highly optimized library (like BLAS) that makes maximal use of vectorization and parallel cores.

The Power of Vectorization

The primary motivation for mini-batches is computational efficiency.



The Minibatch SGD Algorithm

Theorem (Minibatch SGD Algorithm)

Let the batch size be b. At each iteration t:

1. *Sample a minibatch of indices $\mathcal{B}_t \subset \{1, \ldots, n\}$ of size b, typically without replacement.*
2. *Compute the minibatch gradient: $\nabla f_{\mathcal{B}_t}(\mathbf{x}_t) = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla f_i(\mathbf{x}_t)$*
3. *Update the parameters: $\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t - \eta \nabla f_{\mathcal{B}_t}(\mathbf{x}_t)$*

The Minibatch SGD Algorithm

Theorem (Minibatch SGD Algorithm)

Let the batch size be b. At each iteration t:

1. *Sample a minibatch of indices $\mathcal{B}_t \subset \{1, \ldots, n\}$ of size b, typically without replacement.*
2. *Compute the minibatch gradient: $\nabla f_{\mathcal{B}_t}(\mathbf{x}_t) = \frac{1}{b} \sum_{i \in \mathcal{B}_t} \nabla f_i(\mathbf{x}_t)$*
3. *Update the parameters: $\mathbf{x}_{t+1} \leftarrow \mathbf{x}_t - \eta \nabla f_{\mathcal{B}_t}(\mathbf{x}_t)$*

The Best of Both Worlds

- **Computational Efficiency:** It is significantly faster than one-by-one SGD due to vectorization.
- **Statistical Efficiency:** Averaging gradients over a minibatch reduces the variance of the gradient estimate. The standard deviation is reduced by a factor of $\sqrt{b}$ compared to SGD. This leads to a more stable convergence path.

The Final Showdown: Time vs. Loss

Analysis

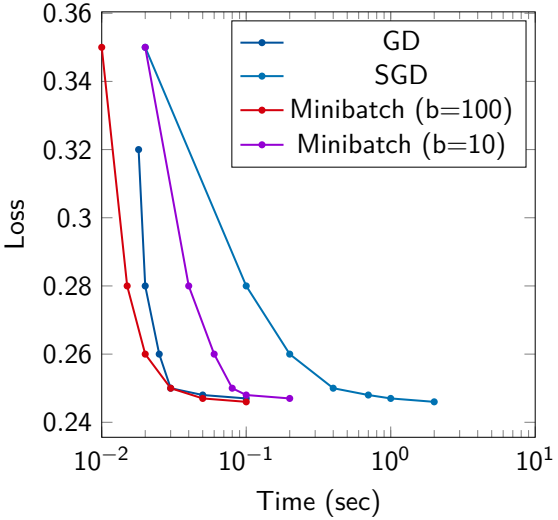
- **Batch:** Each step is very slow, but accurate.
- **SGD:** Converges faster than GD in terms of examples processed, but slow in clock time because it's computationally inefficient.
- **Minibatch:** Offers the best trade-off. It converges quickly and is computationally efficient.

The Final Showdown: Time vs. Loss

Analysis

- **Batch:** Each step is very slow, but accurate.
- **SGD:** Converges faster than GD in terms of examples processed, but slow in clock time because it's computationally inefficient.
- **Minibatch:** Offers the best trade-off. It converges quickly and is computationally efficient.

Minibatch SGD offers the best compromise between statistical and computational efficiency.



Summary: Gradient Descent and its Variants

- **Batch Gradient Descent** uses the full dataset to compute the gradient, resulting in a stable but slow update. The choice of learning rate is critical.

Summary: Gradient Descent and its Variants

- **Batch Gradient Descent** uses the full dataset to compute the gradient, resulting in a stable but slow update. The choice of learning rate is critical.
- **Newton's Method** uses second-order (Hessian) information for faster convergence on convex problems but is computationally infeasible for deep learning and unstable on non-convex problems.

Summary: Gradient Descent and its Variants

- **Batch Gradient Descent** uses the full dataset to compute the gradient, resulting in a stable but slow update. The choice of learning rate is critical.
- **Newton's Method** uses second-order (Hessian) information for faster convergence on convex problems but is computationally infeasible for deep learning and unstable on non-convex problems.
- **Stochastic Gradient Descent (SGD)** uses a single example to compute a cheap but noisy gradient. It requires a decaying learning rate to converge.

Summary: Gradient Descent and its Variants

- **Batch Gradient Descent** uses the full dataset to compute the gradient, resulting in a stable but slow update. The choice of learning rate is critical.
- **Newton's Method** uses second-order (Hessian) information for faster convergence on convex problems but is computationally infeasible for deep learning and unstable on non-convex problems.
- **Stochastic Gradient Descent (SGD)** uses a single example to compute a cheap but noisy gradient. It requires a decaying learning rate to converge.
- **Minibatch SGD** is the practical standard. It balances the trade-off between the stability of GD and the speed of SGD. Its main advantage comes from leveraging the parallel computing power of modern hardware through **vectorization**.

Learning Rate Scheduling

Optimization Strategies

Learning Rate Scheduling

Adjusting the learning rate η during training is often as important as the choice of optimizer. This is called learning rate scheduling.

Why Schedule the Learning Rate?

- **Magnitude:** If η is too large, the optimizer diverges. If too small, it converges too slowly.
- **Decay:** As we approach a minimum, we need to take smaller steps to avoid overshooting and to settle into the minimum. Therefore, the learning rate should generally decay over time.
- **Warmup:** At the start of training, parameters are random. Large steps might be harmful. A "warmup" period with a small initial learning rate that gradually increases can improve stability.

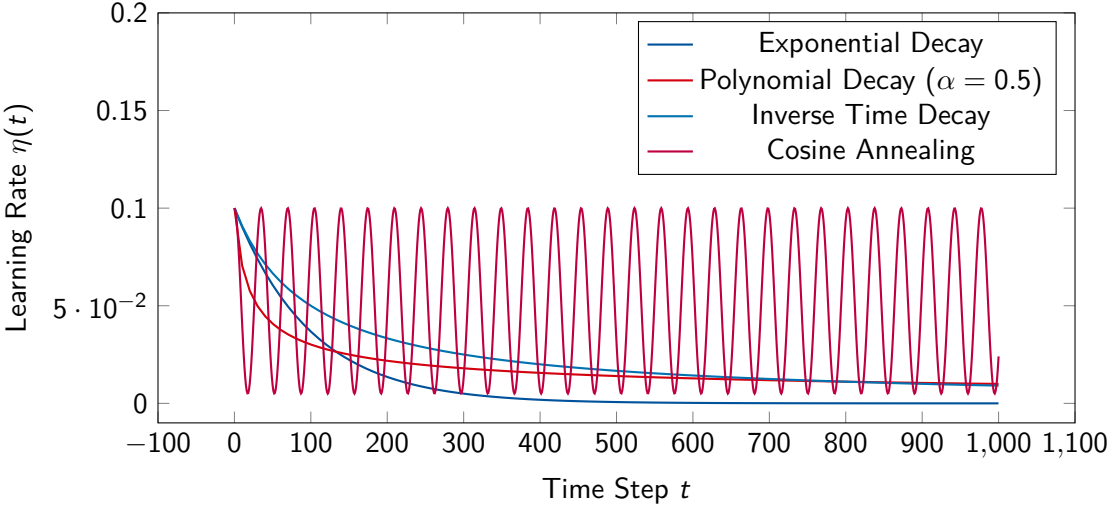
Dynamic Learning Rates (Schedules)

Common Schedules

Let t be the update step number. Common choices for the learning rate schedule $\eta(t)$ include:

- **Piecewise Constant:** $\eta(t) = \eta_i$ if $t_i \leq t < t_{i+1}$. (e.g., "reduce LR by 10x every 30 epochs").
- **Exponential Decay:** $\eta(t) = \eta_0 \cdot e^{-\lambda t}$.
- **Polynomial Decay:** $\eta(t) = \eta_0 \cdot (\beta t + 1)^{-\alpha}$. A popular choice is $\alpha = 0.5$, which gives a decay rate of $O(1/\sqrt{t})$. This rate has good theoretical convergence properties for convex problems.
- **Inverse Time Decay:** A simple scheduler that decreases the learning rate inversly proportional to the current step number. $\eta(t) = \eta_0/(1 + kt)$.
- **Cosine Annealing:** This popular scheduler decreases the learning rate following a cosine curve. $\eta(t) = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{t}{T_{max}}\pi))$.

Dynamic Learning Rates (Schedules)



Accelerated Optimization Methods

Optimization Strategies

The Problem: Oscillations in GD

Recall our ill-conditioned problem: $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$.

Gradient descent struggles:

- The gradient is much larger in the x_2 direction.

The Problem: Oscillations in GD

Recall our ill-conditioned problem: $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$.

Gradient descent struggles:

- The gradient is much larger in the x_2 direction.
- A high learning rate causes divergence in x_2.

The Problem: Oscillations in GD

Recall our ill-conditioned problem: $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$.

Gradient descent struggles:

- The gradient is much larger in the x_2 direction.
- A high learning rate causes divergence in x_2.
- A low learning rate causes painfully slow convergence in x_1.

The Problem: Oscillations in GD

Recall our ill-conditioned problem: $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$.

Gradient descent struggles:

- The gradient is much larger in the x_2 direction.
- A high learning rate causes divergence in x_2.
- A low learning rate causes painfully slow convergence in x_1.
- The optimization path **oscillates** across the narrow valley instead of moving along it.

The Problem: Oscillations in GD

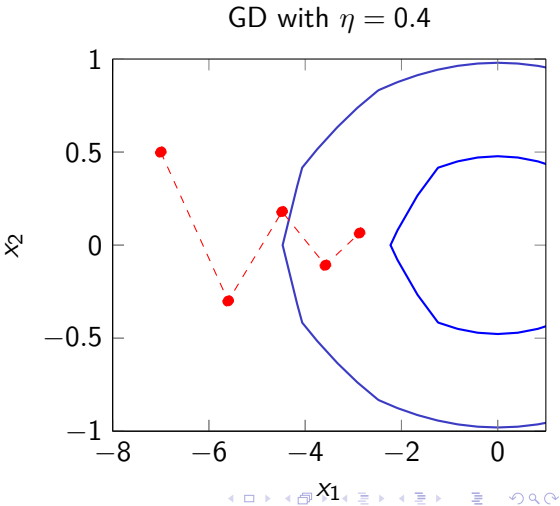
Recall our ill-conditioned problem: $f(x_1, x_2) = 0.1x_1^2 + 2x_2^2$.

Gradient descent struggles:

- The gradient is much larger in the x_2 direction.
- A high learning rate causes divergence in x_2.
- A low learning rate causes painfully slow convergence in x_1.
- The optimization path **oscillates** across the narrow valley instead of moving along it.

The Question

Can we "dampen" these oscillations and "accelerate" progress along the flat direction?



Momentum

Accelerated Optimization Methods

The Momentum Method: A Physical Analogy

The Core Idea

Imagine a heavy ball rolling down the loss surface.

- The ball accumulates **velocity** (i.e., momentum) as it rolls down.

The Momentum Method: A Physical Analogy

The Core Idea

Imagine a heavy ball rolling down the loss surface.

- The ball accumulates **velocity** (i.e., momentum) as it rolls down.
- In steep directions, its velocity changes often, but the oscillations tend to cancel out.

The Momentum Method: A Physical Analogy

The Core Idea

Imagine a heavy ball rolling down the loss surface.

- The ball accumulates **velocity** (i.e., momentum) as it rolls down.
- In steep directions, its velocity changes often, but the oscillations tend to cancel out.
- In flat directions, it consistently accelerates, building up speed.

The Momentum Method: A Physical Analogy

The Core Idea

Imagine a heavy ball rolling down the loss surface.

- The ball accumulates **velocity** (i.e., momentum) as it rolls down.
- In steep directions, its velocity changes often, but the oscillations tend to cancel out.
- In flat directions, it consistently accelerates, building up speed.

We can simulate this by maintaining a "velocity" vector, which is an exponentially weighted moving average of past gradients.

The Momentum Method: A Physical Analogy

The Core Idea

Imagine a heavy ball rolling down the loss surface.

- The ball accumulates **velocity** (i.e., momentum) as it rolls down.
- In steep directions, its velocity changes often, but the oscillations tend to cancel out.
- In flat directions, it consistently accelerates, building up speed.

We can simulate this by maintaining a "velocity" vector, which is an exponentially weighted moving average of past gradients.

Theorem (Momentum Algorithm)

Let $\mathrm{g}_t = \nabla f(\mathrm{x}_t)$. *We introduce a velocity vector* v_t *and a momentum parameter* $\beta \in [0, 1)$.

$$\mathrm{v}_t \leftarrow \beta \mathrm{v}_{t-1} + \mathrm{g}_t \qquad \textit{(Accumulate velocity)}$$

$$\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \eta \mathrm{v}_t \qquad \textit{(Update with velocity)}$$

Understanding Velocity: Leaky Averages

What is the velocity vector v_t really doing? Let's expand the recurrence:

$$
\begin{aligned}
\mathrm{v}_t &= \beta \mathrm{v}_{t-1} + \mathrm{g}_t \\
&= \beta(\beta \mathrm{v}_{t-2} + \mathrm{g}_{t-1}) + \mathrm{g}_t \\
&= \beta^2 \mathrm{v}_{t-2} + \beta \mathrm{g}_{t-1} + \mathrm{g}_t \\
&= \sum_{\tau=0}^{t} \beta^{t-\tau} \mathrm{g}_\tau
\end{aligned}
$$

Understanding Velocity: Leaky Averages

What is the velocity vector v_t really doing? Let's expand the recurrence:

$$\begin{aligned}
\mathrm{v}_t &= \beta \mathrm{v}_{t-1} + \mathrm{g}_t \\
&= \beta(\beta \mathrm{v}_{t-2} + \mathrm{g}_{t-1}) + \mathrm{g}_t \\
&= \beta^2 \mathrm{v}_{t-2} + \beta \mathrm{g}_{t-1} + \mathrm{g}_t \\
&= \sum_{\tau=0}^{t} \beta^{t-\tau} \mathrm{g}_\tau
\end{aligned}$$

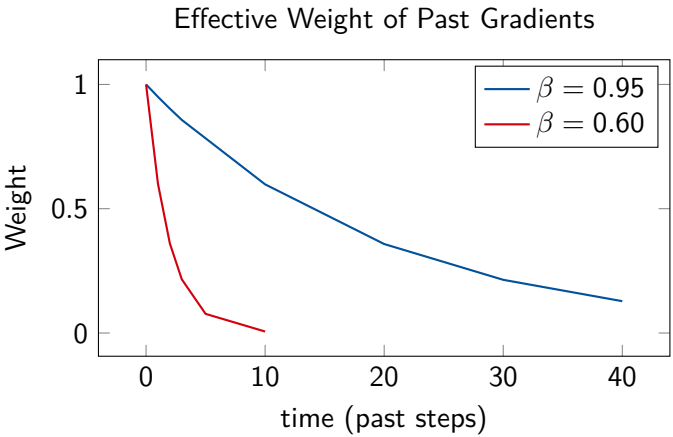
Effective Sample Weight

The velocity v_t is a weighted average of all past gradients. The weights $\beta^{t-\tau}$ decay exponentially, giving more importance to recent gradients.

Understanding Velocity: Leaky Averages

A larger β (e.g., 0.95) means a longer "memory" of past gradients. A smaller β means a shorter memory.

In the limit, the sum of weights is $\sum_{i=0}^{\infty} \beta^i = \frac{1}{1-\beta}$. So the update step is roughly $\frac{\eta}{1-\beta}$ times the averaged gradient.



Momentum in Action

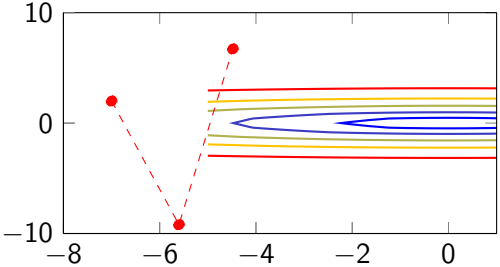
Let's apply Momentum to the same ill-conditioned problem with a high learning rate ($\eta = 0.6$) that caused GD to diverge.

Momentum in Action

Let's apply Momentum to the same ill-conditioned problem with a high learning rate ($\eta = 0.6$) that caused GD to diverge.

Without Momentum ($\beta = 0$)

The path diverges wildly, especially in the steep x_2 direction.



Momentum in Action

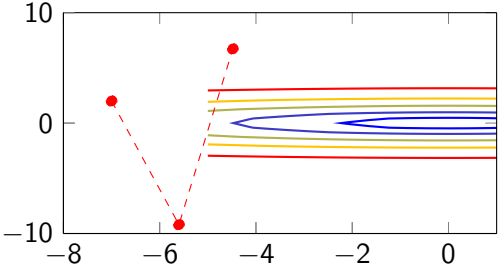
Let's apply Momentum to the same ill-conditioned problem with a high learning rate ($\eta = 0.6$) that caused GD to diverge.

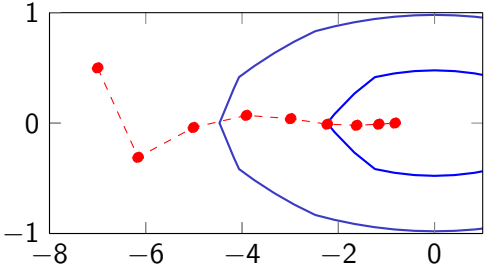
Without Momentum ($\beta = 0$)

The path diverges wildly, especially in the steep x_2 direction.

With Momentum ($\beta = 0.5$)

The path is smoothed out and converges nicely. The oscillations are dampened.





Adagrad

Accelerated Optimization Methods

A New Problem: Sparse Features

Imagine training a language model. The learning rate η decreases over time for all parameters.

Common Words

Words like "the", "a", "is" appear very frequently. Their corresponding embedding vectors receive frequent updates. The decaying learning rate is appropriate for them.

A New Problem: Sparse Features

Imagine training a language model. The learning rate η decreases over time for all parameters.

Common Words

Words like "the", "a", "is" appear very frequently. Their corresponding embedding vectors receive frequent updates. The decaying learning rate is appropriate for them.

Rare Words

Words like "preconditioning" or "eigenvector" appear rarely. Their embeddings receive very few updates. By the time they get a few updates, the global learning rate may have already decayed to a very small value, preventing them from learning effectively.

A New Problem: Sparse Features

Imagine training a language model. The learning rate η decreases over time for all parameters.

Common Words

Words like "the", "a", "is" appear very frequently. Their corresponding embedding vectors receive frequent updates. The decaying learning rate is appropriate for them.

Rare Words

Words like "preconditioning" or "eigenvector" appear rarely. Their embeddings receive very few updates. By the time they get a few updates, the global learning rate may have already decayed to a very small value, preventing them from learning effectively.

The Challenge

We need a mechanism for **per-parameter learning rates**. Parameters that are updated infrequently should have a higher learning rate, and parameters updated frequently should have a lower one.

Adagrad: Adaptive Gradients

The Core Idea

Adagrad adapts the learning rate for each parameter based on the history of its gradients. It does this by dividing the global learning rate by the square root of the sum of all historical squared gradients for that parameter.

Adagrad: Adaptive Gradients

The Core Idea

Adagrad adapts the learning rate for each parameter based on the history of its gradients. It does this by dividing the global learning rate by the square root of the sum of all historical squared gradients for that parameter.

Theorem (Adagrad Algorithm)

Let g_t be the gradient at step t. We maintain a state vector s_t that accumulates squared gradients.

$$\mathrm{s}_t \leftarrow \mathrm{s}_{t-1} + \mathrm{g}_t \odot \mathrm{g}_t \qquad \textit{(Accumulate squared gradient, element-wise)}$$

$$\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \frac{\eta}{\sqrt{\mathrm{s}_t + \epsilon}} \odot \mathrm{g}_t \qquad \textit{(Update with scaled gradient)}$$

Here, $\odot$ is element-wise multiplication and ϵ is a small constant to avoid division by zero.

Adagrad: Adaptive Gradients

Intuition

- If a parameter has consistently large gradients, its entry in $\mathbf{s}_t$ will grow quickly, rapidly decreasing its effective learning rate.
- If a parameter has small or zero gradients (like for a sparse feature), its entry in $\mathbf{s}_t$ grows slowly, keeping its effective learning rate high.

Adagrad: Adaptive Gradients

Intuition

- If a parameter has consistently large gradients, its entry in $\mathbf{s}_t$ will grow quickly, rapidly decreasing its effective learning rate.
- If a parameter has small or zero gradients (like for a sparse feature), its entry in $\mathbf{s}_t$ grows slowly, keeping its effective learning rate high.

On our ill-conditioned problem, Adagrad acts as a **preconditioner**. It automatically corrects for the different scales of the gradients.

The Problem's Geometry

The gradient in the x_2 direction is much larger than in the x_1 direction.

$$\nabla f = [0.2x_1, 4x_2]$$

Adagrad in Action

Adagrad's Solution

- The accumulator s_2 for the second parameter will grow much faster than s_1.
- This will scale down the updates in the x_2 direction more aggressively.
- A more direct path to the minimum, as if the problem were better conditioned.

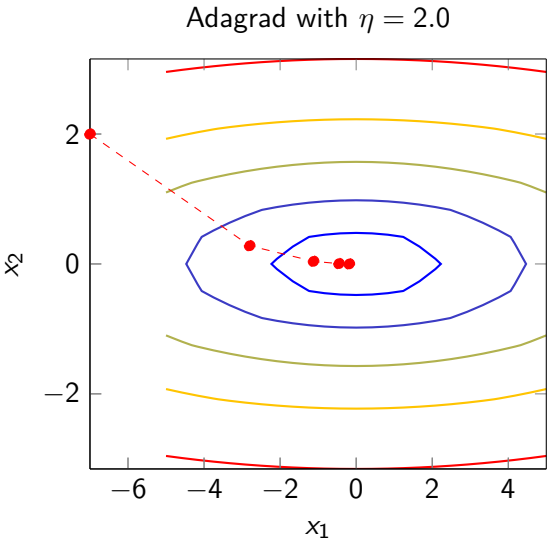
Adagrad in Action

Adagrad's Solution

- The accumulator s_2 for the second parameter will grow much faster than s_1.
- This will scale down the updates in the x_2 direction more aggressively.
- A more direct path to the minimum, as if the problem were better conditioned.

The Drawback

The accumulator s_t only ever increases. After enough time, the learning rate for all parameters will approach zero, stopping training prematurely.



RMSProp

Accelerated Optimization Methods

Fixing Adagrad's Decaying Learning Rate

The Problem with Adagrad

It accumulates squared gradients over the *entire history*, causing the learning rate to eventually vanish.

Fixing Adagrad's Decaying Learning Rate

The Problem with Adagrad

It accumulates squared gradients over the *entire history*, causing the learning rate to eventually vanish.

RMSProp's Solution

- Don't use the whole history! Use an **exponentially weighted moving average** (a leaky average) of the squared gradients, just like Momentum did for the gradients themselves.
- This allows the accumulator to "forget" the distant past.

RMSProp

Theorem (RMSProp Algorithm)

Let g_t *be the gradient. We use a decay rate* $\gamma \in [0, 1)$.

$$\mathrm{s}_t \leftarrow \gamma \mathrm{s}_{t-1} + (1-\gamma)(\mathrm{g}_t \odot \mathrm{g}_t) \qquad \textit{(Leaky average of squared gradients)}$$

$$\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \frac{\eta}{\sqrt{\mathrm{s}_t + \epsilon}} \odot \mathrm{g}_t \qquad \textit{(Update is same as Adagrad)}$$

RMSProp

Theorem (RMSProp Algorithm)

Let g_t be the gradient. We use a decay rate $\gamma \in [0, 1)$.

$$\mathrm{s}_t \leftarrow \gamma \mathrm{s}_{t-1} + (1-\gamma)(\mathrm{g}_t \odot \mathrm{g}_t) \qquad \textit{(Leaky average of squared gradients)}$$

$$\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \frac{\eta}{\sqrt{\mathrm{s}_t + \epsilon}} \odot \mathrm{g}_t \qquad \textit{(Update is same as Adagrad)}$$

RMSProp combines the per-parameter adaptivity of Adagrad with the "forgetting" mechanism of Momentum, making it much more suitable for deep learning.

Adadelta

Accelerated Optimization Methods

Adadelta: No More Learning Rate?

Adadelta is another variant that seeks to solve Adagrad's decaying learning rate problem.

The Core Idea

It removes the need for a global learning rate η altogether. Instead, it adapts the learning rate using information about past *parameter updates*.

Adadelta: No More Learning Rate?

Adadelta is another variant that seeks to solve Adagrad's decaying learning rate problem.

The Core Idea

It removes the need for a global learning rate η altogether. Instead, it adapts the learning rate using information about past *parameter updates*.

Two State Variables

Adadelta maintains two leaky averages:

1. s_t: A leaky average of squared gradients (just like RMSProp).
2. $\Delta \mathrm{x}_t$: A leaky average of squared *parameter updates*.

Adadelta: No More Learning Rate?

Theorem (Adadelta Algorithm)

Let ρ be the decay rate.

1. *Update squared gradient average:* $\mathrm{s}_t \leftarrow \rho \mathrm{s}_{t-1} + (1-\rho)(\mathrm{g}_t \odot \mathrm{g}_t)$
2. *Compute the update:* $\mathrm{g}'_t = \frac{\sqrt{\Delta \mathrm{x}_{t-1} + \epsilon}}{\sqrt{\mathrm{s}_t + \epsilon}} \odot \mathrm{g}_t$
3. *Update squared update average:* $\Delta \mathrm{x}_t \leftarrow \rho \Delta \mathrm{x}_{t-1} + (1-\rho)(\mathrm{g}'_t \odot \mathrm{g}'_t)$
4. *Apply the update:* $\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \mathrm{g}'_t$

Adadelta: No More Learning Rate?

Theorem (Adadelta Algorithm)

Let ρ be the decay rate.

1. *Update squared gradient average:* $\mathrm{s}_t \leftarrow \rho \mathrm{s}_{t-1} + (1-\rho)(\mathrm{g}_t \odot \mathrm{g}_t)$
2. *Compute the update:* $\mathrm{g}'_t = \frac{\sqrt{\Delta \mathrm{x}_{t-1} + \epsilon}}{\sqrt{\mathrm{s}_t + \epsilon}} \odot \mathrm{g}_t$
3. *Update squared update average:* $\Delta \mathrm{x}_t \leftarrow \rho \Delta \mathrm{x}_{t-1} + (1-\rho)(\mathrm{g}'_t \odot \mathrm{g}'_t)$
4. *Apply the update:* $\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \mathrm{g}'_t$

Intuition

The term $\frac{\sqrt{\Delta \mathrm{x}_{t-1}}}{\sqrt{\mathrm{s}_t}}$ approximates a learning rate, effectively self-adjusting based on the recent dynamics of the optimization.

Adam

Accelerated Optimization Methods

Adam: Adaptive Moment Estimation

Adam is arguably the most popular "go-to" optimization algorithm. It combines the best of two worlds:

Momentum + **RMSProp**

Adam: Adaptive Moment Estimation

Adam is arguably the most popular "go-to" optimization algorithm. It combines the best of two worlds:

Momentum + **RMSProp**

The Components

- It keeps an exponentially weighted moving average of past gradients (**first moment estimate**), similar to Momentum's velocity. Let's call this v_t.

Adam: Adaptive Moment Estimation

Adam is arguably the most popular "go-to" optimization algorithm. It combines the best of two worlds:

Momentum + **RMSProp**

The Components

- It keeps an exponentially weighted moving average of past gradients (**first moment estimate**), similar to Momentum's velocity. Let's call this v_t.
- It keeps an exponentially weighted moving average of past *squared* gradients (**second moment estimate**), similar to RMSProp's accumulator. Let's call this s_t.

Adam: Adaptive Moment Estimation

Adam is arguably the most popular "go-to" optimization algorithm. It combines the best of two worlds:

Momentum + **RMSProp**

The Components

- It keeps an exponentially weighted moving average of past gradients (**first moment estimate**), similar to Momentum's velocity. Let's call this v_t.
- It keeps an exponentially weighted moving average of past *squared* gradients (**second moment estimate**), similar to RMSProp's accumulator. Let's call this s_t.

The Result

Adam computes adaptive, per-parameter learning rates based on the second moment (s_t) and uses the first moment (v_t) as the search direction.

The Adam Algorithm

Theorem (Adam Algorithm)

Let β_1, β_2 be decay rates for the first and second moments.

1. *Update biased first moment:* $\mathrm{v}_t \leftarrow \beta_1 \mathrm{v}_{t-1} + (1 - \beta_1)\mathrm{g}_t$

The Adam Algorithm

Theorem (Adam Algorithm)

Let β_1, β_2 be decay rates for the first and second moments.

1. *Update biased first moment:* $\mathrm{v}_t \leftarrow \beta_1 \mathrm{v}_{t-1} + (1 - \beta_1)\mathrm{g}_t$
2. *Update biased second moment:* $\mathrm{s}_t \leftarrow \beta_2 \mathrm{s}_{t-1} + (1 - \beta_2)(\mathrm{g}_t \odot \mathrm{g}_t)$

The Adam Algorithm

Theorem (Adam Algorithm)

Let β_1, β_2 be decay rates for the first and second moments.

1. *Update biased first moment:* $\mathrm{v}_t \leftarrow \beta_1 \mathrm{v}_{t-1} + (1-\beta_1)\mathrm{g}_t$
2. *Update biased second moment:* $\mathrm{s}_t \leftarrow \beta_2 \mathrm{s}_{t-1} + (1-\beta_2)(\mathrm{g}_t \odot \mathrm{g}_t)$
3. ***Correct for bias:*** *The estimates v_t and s_t are initialized at 0, making them biased towards zero early in training. Adam corrects for this:*

$$\hat{\mathrm{v}}_t = \frac{\mathrm{v}_t}{1-\beta_1^t} \quad \textit{and} \quad \hat{\mathrm{s}}_t = \frac{\mathrm{s}_t}{1-\beta_2^t}$$

The Adam Algorithm

Theorem (Adam Algorithm)

Let β_1, β_2 be decay rates for the first and second moments.

1. *Update biased first moment:* $\mathrm{v}_t \leftarrow \beta_1 \mathrm{v}_{t-1} + (1-\beta_1)\mathrm{g}_t$
2. *Update biased second moment:* $\mathrm{s}_t \leftarrow \beta_2 \mathrm{s}_{t-1} + (1-\beta_2)(\mathrm{g}_t \odot \mathrm{g}_t)$
3. ***Correct for bias:*** *The estimates v_t and s_t are initialized at 0, making them biased towards zero early in training. Adam corrects for this:*

$$\hat{\mathrm{v}}_t = \frac{\mathrm{v}_t}{1-\beta_1^t} \quad \textit{and} \quad \hat{\mathrm{s}}_t = \frac{\mathrm{s}_t}{1-\beta_2^t}$$

4. *Compute the update:*

$$\mathrm{x}_{t+1} \leftarrow \mathrm{x}_t - \eta \frac{\hat{\mathrm{v}}_t}{\sqrt{\hat{\mathrm{s}}_t + \epsilon}}$$

Common values are $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\eta = 0.001$.

Summary

Key Takeaways

- Optimization in deep learning aims to minimize generalization error, not just training error.
- Deep learning landscapes are non-convex, with challenges like local minima and saddle points.
- **Minibatch SGD** is the workhorse, balancing computational efficiency and stable updates.
- **Momentum** accelerates convergence by accumulating velocity in consistent gradient directions.
- **Adaptive methods (AdaGrad, RMSProp, Adam)** use per-parameter learning rates, adapting to data sparsity and problem geometry.
- **Adam** is a robust, default choice that combines momentum and adaptive scaling.
- **Learning rate schedules** (decay, warmup) are critical for achieving state-of-the-art performance and ensuring stable training.